UrROM

How to read, edit, compare and log the Bosch Motronic M2.3 / M2.3.2 chips in the Audi 5-cylinder 20V turbo

3B · RR · S2 (447/857/895 907 404) and AAN · ABY · ADU · RS2 (4A0/8A0 907 551), with KWPBridge live data

Everything in this guide that is a fact about the ECU was traced in the firmware and is written up in the **docs/** folder of the repository; the app shows the same evidence on every cell. Where something is still an assumption it says so, in the app and here.

1. What UrROM is

UrROM opens the EPROM images of the two chips in these ECUs, shows every calibration map the firmware actually references, lets you edit them in real units, keeps the checksum the ECU checks at boot, and writes burner-ready images. It also compares chips, decodes the coding plug, applies known firmware patches, and, with KWPBridge on a K-line cable, overlays the live engine position on the maps.

Supported chips

Family	ECU part numbers	Chips	Notes
M2.3 "404"	447 907 404 (3B, 200 20V), 857 907 404 (RR, UrQuattro), 895 907 404 (S2 3B)	32 KB fuel/ignition + 8 KB boost MCU	16-bit checksum at 0x7F00; early KW1281 dialect
M2.3.2 "551"	4A0 907 551 (AAN/ABY), 8A0 907 551 (ADU/RS2), 895 907 551 (S2 ABY)	64 KB split-bank fuel/ignition + 32 KB boost	firmware low / calibration high; prjmod + 034 tunes
V8	441/077 907 557, 441 907 404 (PT/ABH)	single chip	preliminary map data

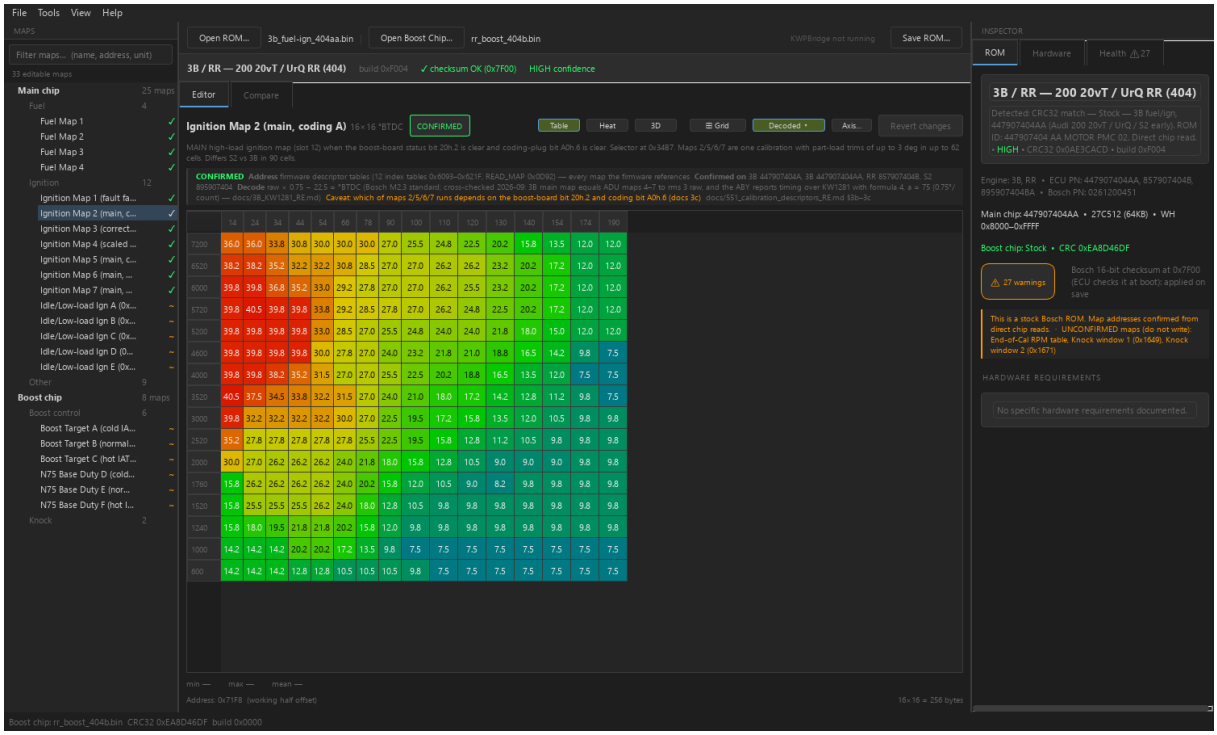
Install and run

Download **UrROM.exe** from the repository's latest release, or run from source:

```
pip install -r requirements.txt
python app/main.py
```

Windows, Python 3.11 or newer. PyQt5, matplotlib and numpy come with the requirements; no OpenGL is needed.

2. The workbench



The workbench: map tree on the left, editor in the middle, inspector on the right. The file bar shows both chips, the checksum state and the KWPBridge status.

• **File** → **Open ROM…** opens the fuel/ignition chip (.bin, .034, 27C512 images fold automatically). **Open Boost Chip…** pairs the boost chip; a known pair is checked against the catalogue.

• **Map tree**: fuel, ignition, other, boost control, knock. Click a map to edit it. The tick marks the confidence (confirmed, provisional, unconfirmed).

• **Editor**: the map with its real axes (rpm down, load across; TPS × rpm on the boost chip). The description, the provenance line and the confidence badge sit above it.

• **Inspector**: ROM identity (variant, part numbers, CRC, build), the Hardware tab, and the Health scan.

• **View** menu: dock layout is saved between sessions.

Saving

Save ROM… writes the edited chip. On 3B / RR / S2 chips the 16-bit checksum at 0x7F00 is recomputed (the ECU verifies it at boot and logs a fault if it is wrong); on prjmod 551 chips the 0x3FFA checksum is. **File** → **Write 27C512 images** writes the 64 KB image a modern burner needs: a 32 KB chip stored twice, an 8 KB boost chip eight times. Opening such an image folds it back.

> Only 27C512 EPROMs are still easy to buy, so burn the 27C512 image, not the native file.

3. Editing a map

Type a value in a cell. Decoded units are the default (°BTDC, kPa, %); the **Decoded** button switches to raw bytes. Right-click for undo, fill, scale and copy/paste. **Revert changes** restores the map.

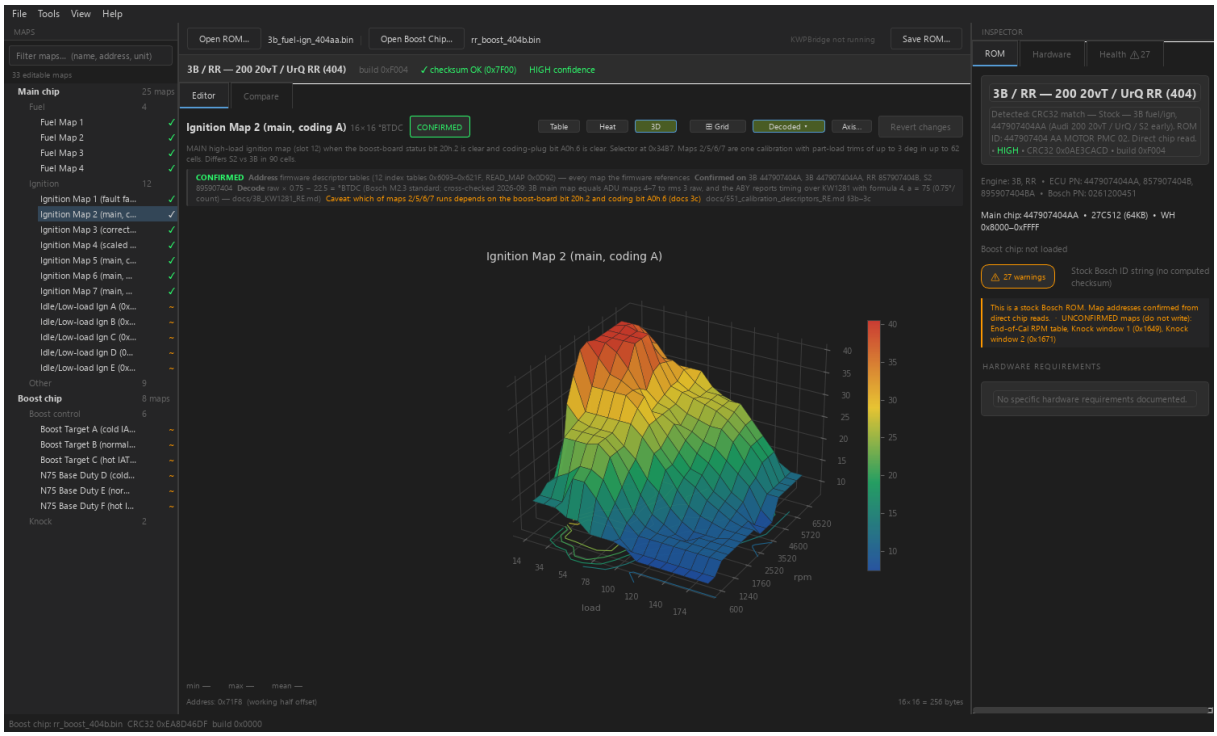
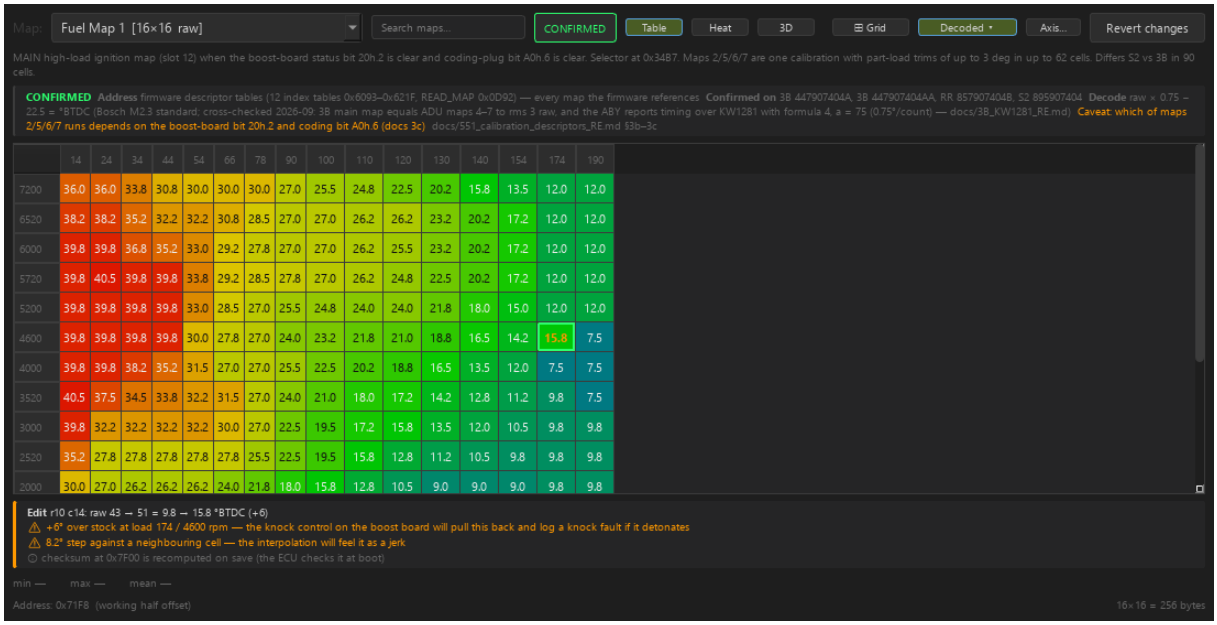


Table / Heat / 3D on every map. Drag the surface to orbit. Edits, the raw toggle, the boost sensor scale and the live cursor all follow into the views.

Guard rails

Every edit gets an **Edit** line under the map: the change in raw and in real units, then anything the firmware traces say it risks. Flagged cells turn amber or red and carry the same text in their tooltip. Nothing is blocked; you decide.

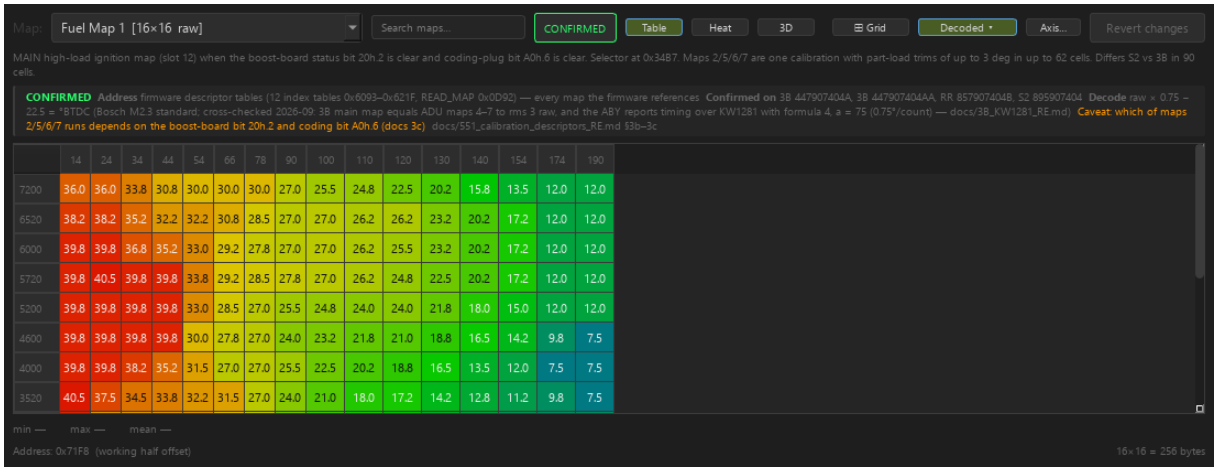


An ignition edit at load 174 / 4600 rpm: +6° over stock in the knock region, and an 8° step against a neighbour.

- Boost target within a few counts of the sensor's full scale (255 is the sensor ceiling; the controller needs headroom).
- Added advance at load ≥ 130 and ≥ 3500 rpm: the boost board's knock control will pull it back and log a fault.
- A step of 6° or more against a neighbouring cell.
- Fuel taken out under boost; the last load column, which runs at every load above the axis.
- A typed value the byte cannot hold; the checksum that will be rewritten on save.

Provenance

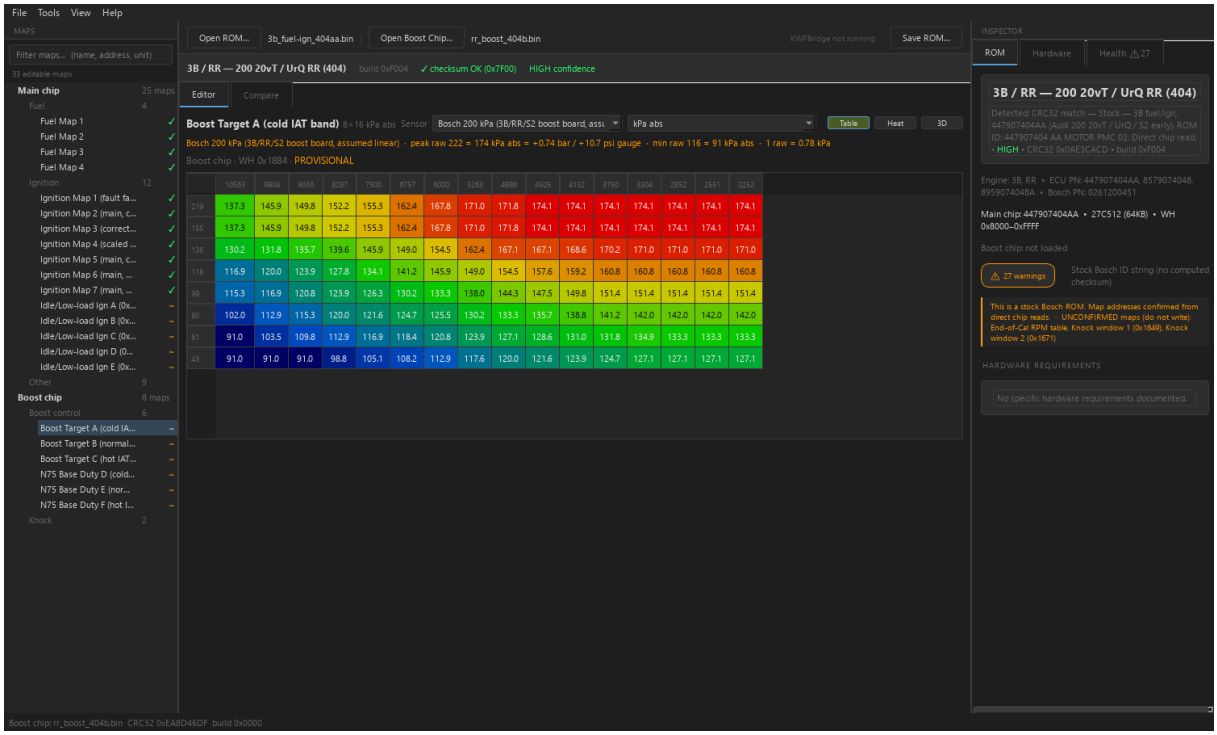
The line under each map's description, and the tail of every cell's tooltip, say where the address came from (firmware descriptor tables, an XDF, or a diff), which real chips confirmed it, the decode formula and where it was established, the axis source, any open caveat, and the docs section to read.



Provenance for the 3B main ignition map: descriptor tables at 0x6093–0x621F, confirmed on four chips, raw × 0.75 – 22.5 °BTDC.

4. The boost chip

The boost MCU holds three Boost Target tables (selected by intake-air band) and three N75 duty tables, TPS down, rpm across. A table byte is a sensor voltage, so pressure depends on the fitted sensor: pick it in the **Sensor** selector and choose kPa absolute, bar gauge or psi gauge.



Boost Target A on the RR chip at the assumed 200 kPa scale: the peak line shows raw, kPa, bar and psi.

Which sensor?

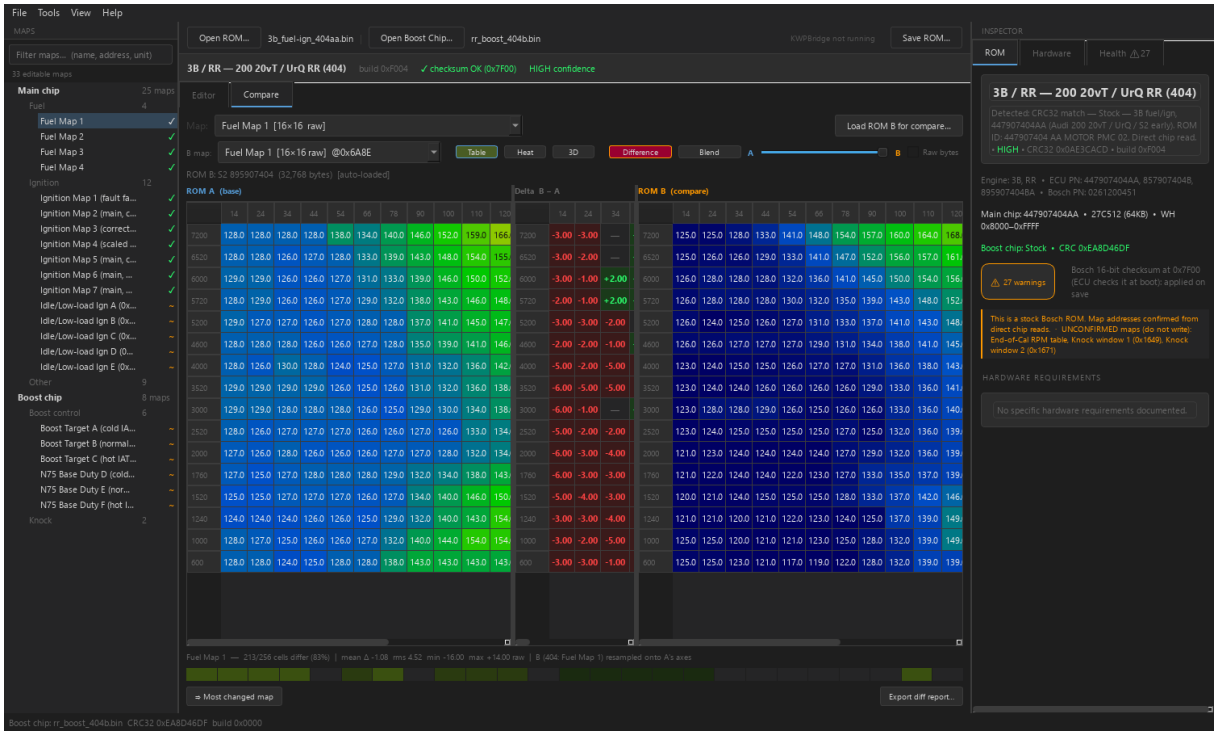
Sensor	Vout at atmosphere (key on, engine off)	Fitted on
Bosch 200 kPa (linear, assumed)	2.50 V	3B / RR / S2 boost board (stock)
MPX4250A 250 kPa	1.80 V	AAN / ABY stock, QLCC
linear 250 kPa	2.00 V	aftermarket 250 kPa swaps
linear 300 kPa	1.67 V	RS2 R201 swap, 034EFI "3 bar" tunes
MPXH6400A 400 kPa	1.17 V	prjmod speed-density

Identify from a voltage reading… in the selector ranks the sensors from a measured output voltage. The controller itself does not depend on the scale: it subtracts its own key-on ambient from both target and measurement, so the tables are absolute sensor counts and the scale only changes what a count means. That is also why a 250 kPa sensor raises boost without any table edit: the same bytes sit 145 counts above ambient instead of 109.

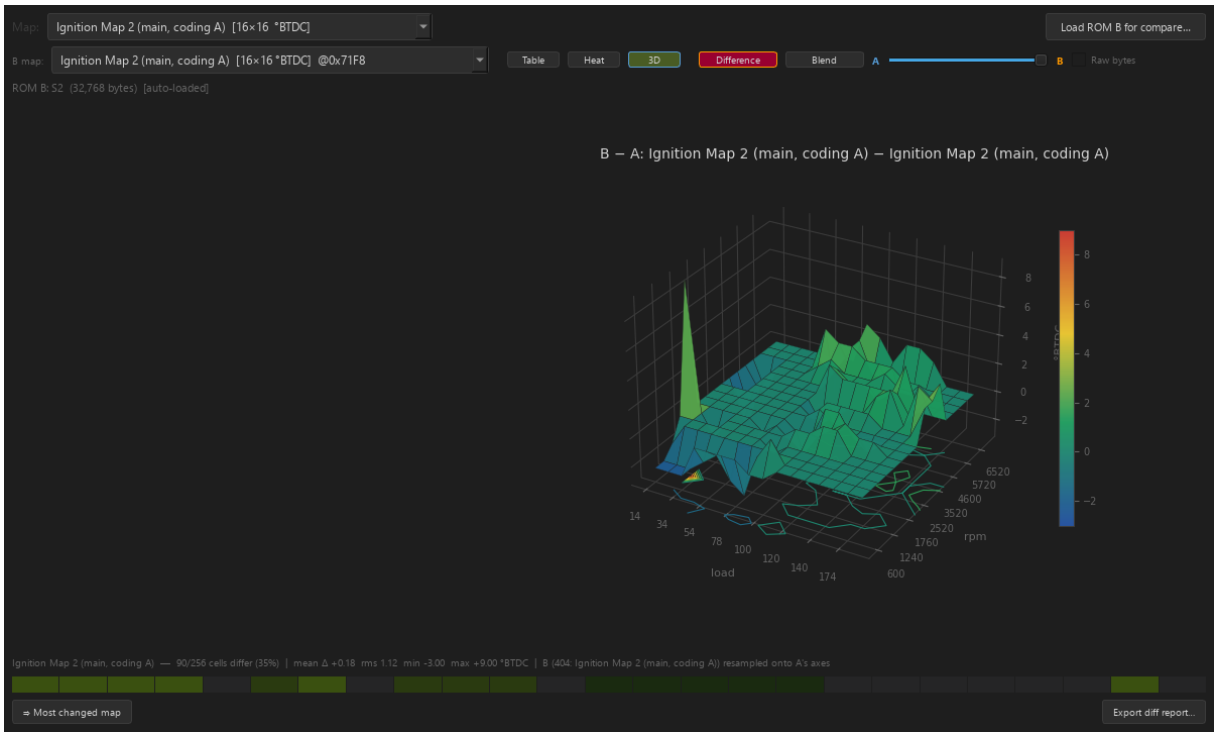
> The stock 3B board's sensor is still assumed to be linear 0–200 kPa. A mechanical gauge reading on one full pull confirms or corrects it; the cluster gauge cannot, because its scaling is unknown.

5. Comparing chips

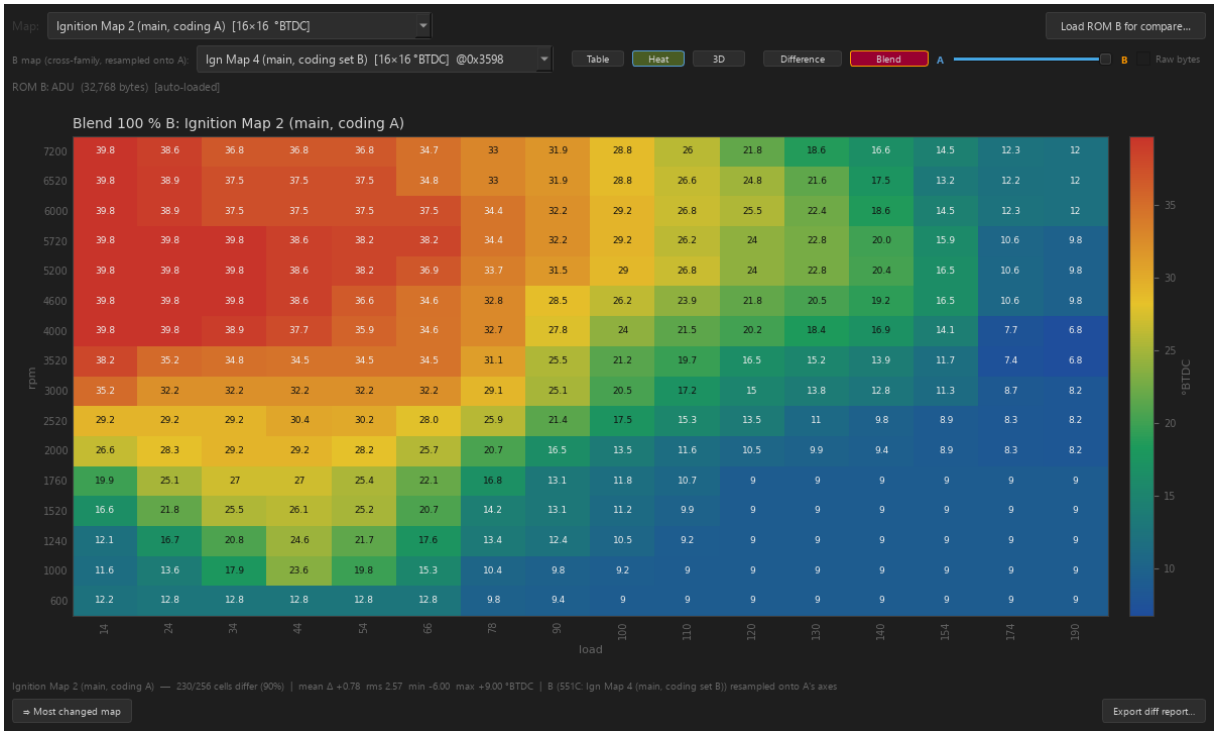
The **Compare** tab takes a second chip, even from the other family. A 551 map is paired with the 3B map by role and bilinear-resampled onto the A chip's axes, so the delta is in real units on your grid. Same-family chips at the same address compare cell for cell.



The Compare tab with the S2 chip loaded as ROM B against the 3B: base, delta and compare tables side by side, the per-map change strip below, and Most changed map to jump to the biggest difference. Fuel Map 1 differs in 213 of 256 cells, all a few raw counts leaner on the S2.



Difference surface: S2 minus 3B ignition. The overrun spike and the mid-band ridge are the S2's part-throttle advance.

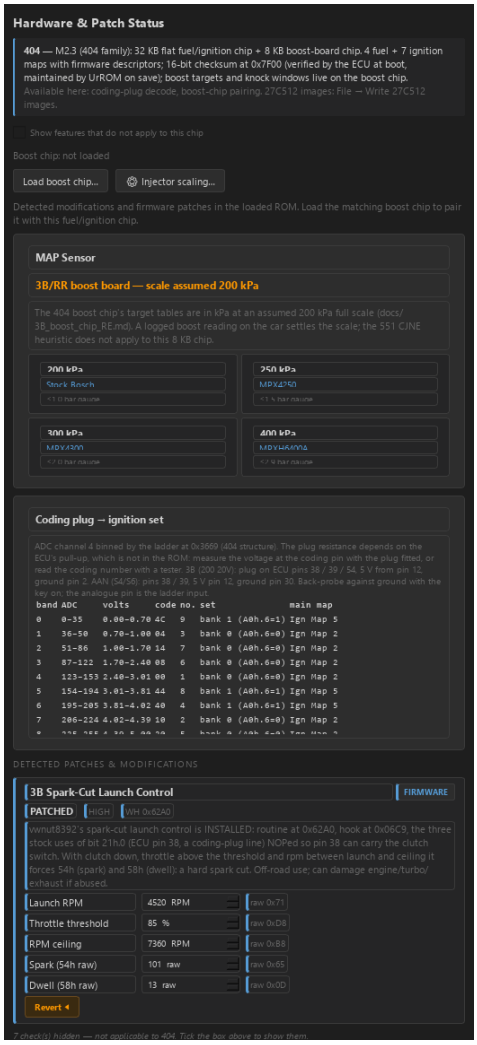


Blend mode with the slider at 100 %: the ADU (RS2) main ignition map resampled onto the 3B's load and rpm axes.

- **B map** chooses which map on the B chip to pair; the default follows the role (fuel, main ignition).
- **Difference** shows B – A; **Blend** morphs A into B with the slider.
- **Raw bytes** compares undecoded values when a formula is in question.
- The summary gives cells differing, mean, rms, min and max; **Export diff report…** and the **Most changed map** jump work for same-family chips.

6. The Hardware tab

Everything about the chip that is not a map: what the chip family supports, the boost sensor, the coding plug, firmware patches and modifications. Items that cannot apply to the loaded chip are hidden; a checkbox shows them dimmed.



A 3B with vwnut8392's launch control installed: the patch card with its five editable values, and the coding-plug decoder above it.

Coding plug

Both families read the coding plug on ADC channel 4 and bin it into nine bands. The card shows each band's voltage, the coding number a tester would display, and the ignition set it selects. The plug's resistance is not in the ROM: measure the pin voltage with the plug fitted (3B: ECU pins 38 / 39 / 54 against pin 2; AAN: 38 / 39 against pin 30) or read the coding number from the ECU's identification. The 3B sends it in the third ID block as "PMC n".

Firmware patches

Patch	Chips	What it does
3B Spark-Cut Launch Control (vwnut8392 V1.01)	3B, RR, S2	Hard spark cut with clutch down, throttle over the threshold, rpm between launch and ceiling. Needs ECU pin 38 wired to a clutch switch. Launch rpm, throttle, ceiling, spark and dwell are editable.
MFTS boost-cut bypass, load-overflow decap, lambda cold-start delay	551 (prjmod)	Existing 551 patches; offsets confirmed per variant.

Apply and Revert restore the exact stock bytes; UrROM keeps the factory ID text so a tester still sees a stock ECU. Launch control is off-road use and can damage the engine, turbo or exhaust if abused.

7. Live data with KWPBridge

KWPBridge (github.com/dspl1236/KWPBridge) owns the K-line on a KKL cable and broadcasts the ECU's data on a local port. UrROM connects to it automatically, checks the ECU's part number against the loaded chip, and then draws the live position on the current map.

```
python -m kwpbridge --port COM3 --protocol kwp1281
```

- 551 family: the standard four-cell groups (rpm, load, coolant, lambda, timing, MAP on prjmod).
- 3B / RR / S2: the early KW1281 dialect. KWPBridge reads the ten-byte group 000 and a RAM window that gives rpm and load in the map's own axis units, plus the ECU's own timing formula.
- The badge in the file bar shows the connection state and the live summary.

Recording and trace

Tools → **Start recording live data…** writes every sample to a CSV that **Overlay data log on map…** replays onto any map. **Live trace on current map** paints where the engine actually runs.



The live trace on the fuel map: ringed cells are where the engine spent its time, sized by sample count. The table lightens the same cells and gives the count in the tooltip.

Bench mode

Tools → **Bench mode (simulated engine)** starts KWPBridge's mock ECU for the loaded chip inside UrROM. The cursor walks the maps through cold start, idle, cruise, a boost run and decel, the trace fills in,

and the badge reads BENCH. Use it to learn the maps or rehearse a session before the cable goes on.



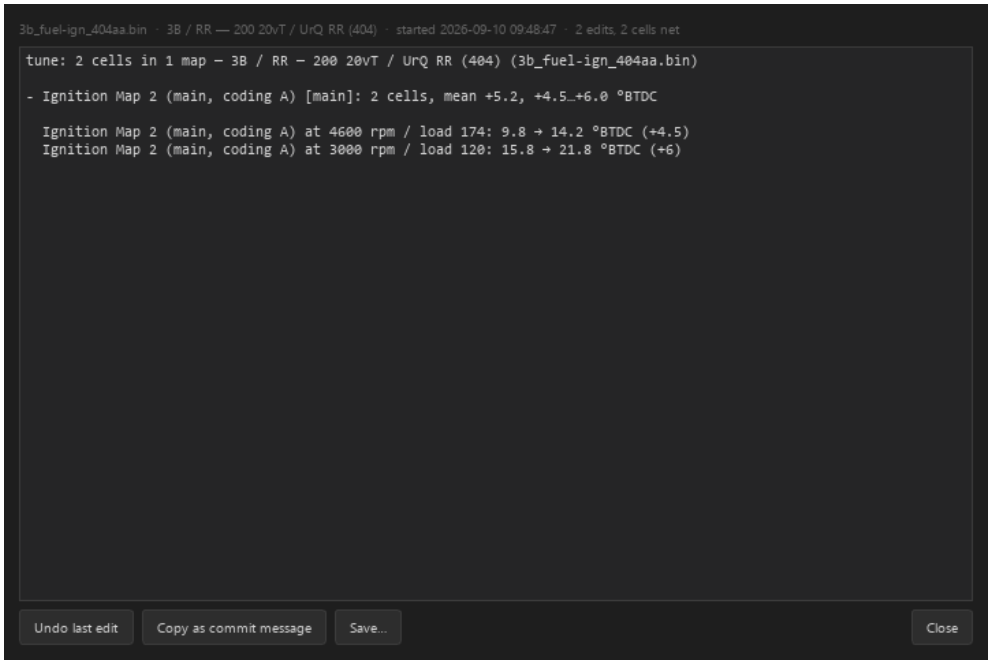
Bench mode on a 3B, Heat view: the file bar reads BENCH with the simulated rpm, coolant, lambda and timing, the amber square is the live cursor and the white rings are the trace filling in as the mock walks Fuel Map 1 through warm-up exactly as a car would.

8. Session log and undo

Tools → **Session log… (Ctrl+L)** lists every edit as a sentence with its axis position and real units, one line per cell (edits back to the starting value drop out):

```
Ignition Map 2 (main, coding A) at 4600 rpm / load 174: 9.8 → 15.8 °BTDC (+6)
Boost Target B (normal IAT band, default) at TPS 219 / 4886 rpm: 189.8 → 196.1 kPa abs (+6.3)
```

- **Copy as commit message**: a git-style title, a per-map summary and the sentences.
- **Undo last edit** reverts the most recent change in whichever tab holds that map.
- **Save…** writes the log as text or HTML. Boost-chip edits are logged in the sensor's units.



The session log after two ignition edits: the header shows the file, chip and start time, the body is the commit-style summary and one sentence per cell, and the buttons undo, copy or save it.

Command line

```
python -m urrom.cli scan rom.bin # tuning checks, exit 2 on errors
python -m urrom.cli info rom.bin # identification
python -m urrom.cli maps rom.bin # every map the firmware references
python -m urrom.cli chip boost.bin # 8/32 KB image → 27C512 image (--to native folds
back)
python -m urrom.cli xcompare A.bin B.bin --role ign # cross-family compare
python -m urrom.cli coding rom.bin --volts 2.5 # coding-plug band lookup
```

9. Recipes for a 3B (200 20V)

OEM combination

The three factory chip sets for the 3B engine differ in known ways. The RR boost chip runs about +5 kPa with more wastegate duty at high rpm; the S2 fuel/ignition chip adds 1.5–3° in the mid band and top-end fuel while leaning idle and cruise; the S2 boost chip is 0.2 bar lower. The combination that moves the right things is the **RR boost chip with the S2 fuel/ignition chip**, every byte factory. Images are in **roms/27c512/404/**.

Repository layout: stock chip reads sit flat in **roms/**; UrROM-built tunes are filed by ECU family under **roms/tunes/<family>/** and their burner images under **roms/27c512/<family>/**, where **404** is the 3B / RR / S2 fuel-ign and boost chips and **551** is AAN / ABY / ADU / RS2. Every tune has a row in roms/README.md with its base chip, what changed, its CRC32 and its status.

Boost chip	Target B peak (200 kPa scale)	gauge
3B stock	186 kPa	+0.86 bar / 12.5 psi
RR	191 kPa	+0.91 bar / 13.2 psi
UrROM stage 1 (RR + 6 raw)	196 kPa	+0.96 bar / 13.9 psi
S2	169 kPa	+0.69 bar / 10.0 psi

Stage 1

roms/tunes/404/3b_stage1_boost_rrbase_plus5kpa.bin is the RR boost chip with the three target tables lifted 6 raw and capped at 250. Duty, gains, knock tables and ceilings are untouched. Run it with a boost gauge: about 14 psi confirms the sensor scale and that the controller reaches its target.

The stock sensor caps any 200 kPa-sensor chip near 1.0 bar (raw 255 is full scale). Beyond that the first step is a 250 kPa sensor, identified by its voltage at atmosphere, after which the same tables mean about 1.4 bar and must be scaled back down in the editor.

Stock boost on a 3-bar sensor

The main ECU reads the MAP sensor in one place only, a 3-point correction that is 1.00 on every stock chip, so a different sensor changes nothing but the boost board. **python -m urrom.cli boost-sensor <chip> <out> --to vmap300_034** re-encodes a stock boost chip so every target, threshold, ceiling and correction keeps its kPa; duty tables are untouched. Two are published for the GM / 034 3-bar sensor (kPa = raw/255 × 300 + 21):

File (roms/tunes/404/)	Base	Peak target	Use
3b_boost_404aa_3bar034.bin	stock 3B boost	12.4 psi (stock)	3-bar sensor, otherwise stock car
rr_boost_404b_3bar034.bin	stock RR boost	13.3 psi (RR)	3-bar sensor with the RR profile

Bench the sensor before burning: 5 V supply, about 1.3 V at atmosphere for a GM 3-bar; the identifier in the boost tab confirms the preset from that number. The cluster boost gauge is driven off the boost board with an untraced scale and will read about 1.5 × high on a 3-bar sensor; use a mechanical gauge. One raw count is now 1.18 kPa instead of 0.78, so the loop acts 1.5 × harder per kPa; if boost hunts on a steady pull, rebuild with --scale-gains.

Wiring an external 3-bar sensor

The stock sensor sits on the boost board inside the ECU case with a hose nipple through the wall. An external GM 3-bar is three wires from that board's sensor pads out through the case, plus its own vacuum line.

- **Ground and 5 V from the boost board's sensor pads**, not from a chassis ground or the coding-plug 5 V at the connector: the signal is 0–5 V read against the board's own reference, and a 0.1 V ground offset is 6 kPa.
- **Short and away from ignition.** A metre of twisted three-core is fine; shield it if it must run near the coil or plug leads, shield grounded at the ECU end only.
- **GM 3-bar pins**: the three-pin Delphi connector is ground / signal / 5 V on A / B / C — verify on the datasheet for your part, some aftermarket copies swap the outer two.
- **Vacuum**: tee into the line that feeds the ECU nipple today (after the throttle body); keep the hose short and add a restrictor if the live boost trace looks noisy.
- **Bench it first**: sensor on 5 V from a bench supply, meter on the signal, about 1.3 V at atmosphere. Enter that in the boost tab's identifier and it should pick the 034 3-bar preset.
- Leave the old board sensor in place unpowered and move only the signal, or remove it; the first is easier to revert.

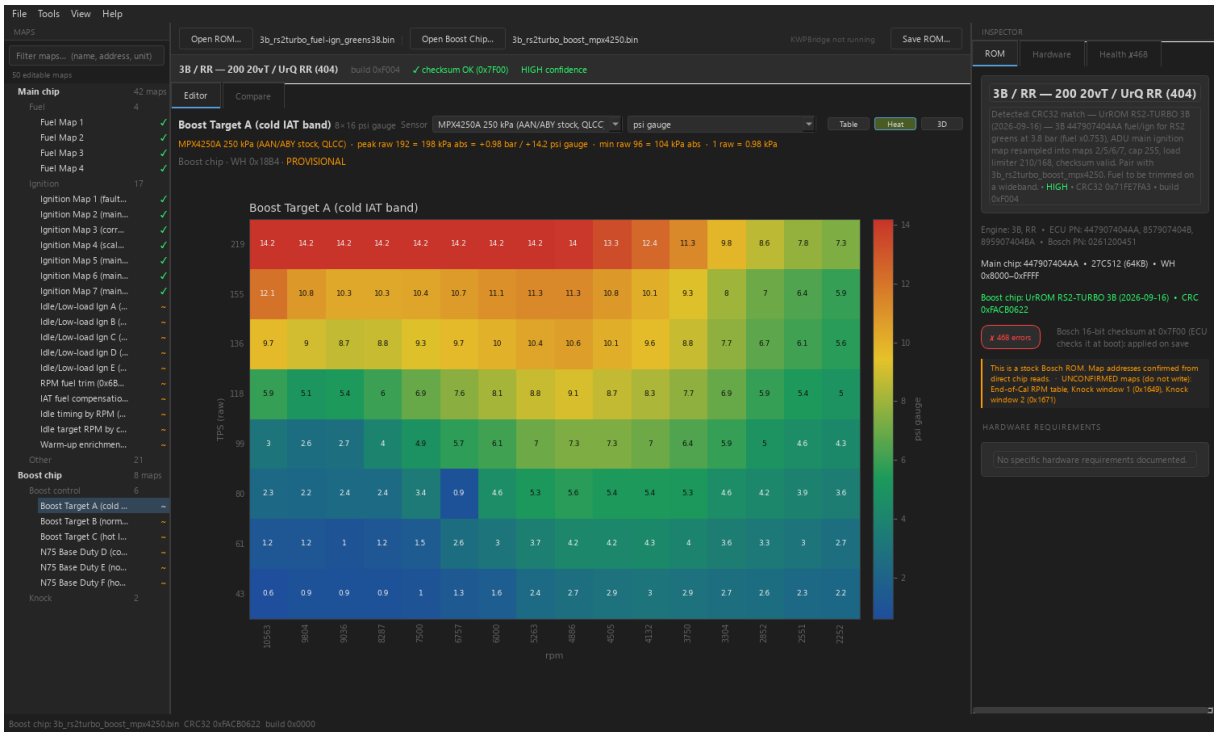
The RS2-turbo chipset

What the 200 20V would have shipped with if it had the RS2's K24-7200: **roms/tunes/404/3b_rs2turbo_boost_mpx4250.bin** and **3b_rs2turbo_fuel-ign_greens38.bin**, built by **python -m urrom.cli rs2-chipset** from the 3B's own chips with the RS2 D02 boost chip and the ADU fuel/ign chip as references. Hardware assumed: K24-7200, RS2 green injectors (0 280 150 984, 405 cc)

on the RS2 3.8 bar regulator, and the AAN/ADU 250 kPa MPX4250A sensor on the boost board.

- **Boost chip:** the 3B chip re-encoded for the 250 kPa sensor (the RS2's 14.2 psi peak is raw 253 of 255 on the 200 kPa sensor), the RS2 factory full-throttle curve written in by rpm with the 3B's part-throttle shape scaled under it, the RS2 per-rpm N75 duty envelope, duty ceiling 78 %, overboost release 22 psi.
- **Fuel/ign chip:** fuel maps and cranking × 305/405 for the greens, the ADU main ignition map resampled onto the 3B axes into all four main maps, air-per-rev cap 255, load limiter 210, checksum valid.
- **Not RS2 on purpose:** the fuel map. The ADU's runs 15 % leaner above load 100 relative to an injector scale the 551 keeps elsewhere, so it is 3B × injector ratio here, to be trimmed on a wideband.

rpm	3B WOT target	RS2 WOT target (in the chip)
2250	12.5 psi	7.3 psi
3300	12.5	9.8
4100	12.5	12.4
4900	12.3	14.0
5300 and up	12.2 → 10.3	14.2



The RS2-turbo boost chip in the boost tab on the MPX4250A preset in psi: the RS2 curve in the top TPS row (7.3 psi at 2250 rpm rising to 14.2 from 5000), the 3B's part-throttle rows scaled beneath it. Rpm runs right to left as the boost MCU stores it.

Order on the car: greens and regulator with the old boost chip first (idle and cruise lambda), then the 250 kPa sensor with this boost chip (about 1.5 V at atmosphere on the bench), then the first pulls with a wideband, checking the loop reaches 14 psi by 5000 rpm without the release tripping. docs/3B_RS2_turbo_chipset.md has the numbers.

10. What to log, and when

Every tool in the repository that is still waiting on something is waiting on a log. KWPBridge's 3B profile reads **group 100**, a RAM window that carries rpm at 40 rpm per count and LOAD in the unit the map axes use; group 000's rpm byte saturates at 2550 rpm, so it serves idle only. **Group 101** is the MAF window. In UrROM, Tools → Start recording live data writes the CSV and paints the trace on the open map.

Situation	Record	Decides
Hybrid chip (3B fuel, S2 ign, RR boost), first drives	group 100 at a steady 2000 rpm cruise and a gentle roll-on from 1800; whether the stumble is there	daily chip, or the 2k region of the fuel map is the next edit
Any chip, third gear, 2000 to 6500 rpm full throttle	group 100: LOAD and rpm	where LOAD sits against the 200 cap; sets the GT3071 load-scale factor (a guess until then)
GM 3-bar MAP sensor, key on, before any chip is burned	output voltage at atmosphere	about 1.3 V = vmap300_034 preset; about 1.0 V = zero-offset part, lin300
The present chip, any drive	fault memory, boost-gauge peak, coding number	the baseline every later change is compared with
Bosch 550 injectors on the 550 chip	wideband lambda at idle and 3000 rpm cruise, warm, plus group 100	the fuel trim on top of the 305/550 scaling
3-bar sensor on the stock-kPa boost chip	gauge peak on a pull, against the baseline	that the sensor conversion is right before any target is raised
3-bar sensor, targets raised in steps	gauge peak, wideband on the pull, group 100	each step; stop when lambda moves or knock retard shows

Name the file with the chip and the situation, note the coding number and the ambient temperature, and for a pull the gear and rpm range. Several short clean segments beat one long file: the fits use medians per rpm bin. docs/3B_what_to_log.md has the full list and says which tool each number feeds.

Where to read more

Document	What it holds
docs/3B_boost_chip_RE.md	boost MCU tables, axes, knock evaluator, control loop, sensor scale
docs/551_calibration_descriptors_RE.md	descriptor tables, ignition selection (3B §3c, 551 §3e), coding plug §3f
docs/3B_KW1281_RE.md	the 3B's K-line dialect and group 000 decodes
docs/3B_launch_control_RE.md	the launch-control patch, byte by byte, and the 404 checksum
docs/AAN_3B_port_notes.md	AAN/ABY vs 3B comparison, prj file verdict
docs/3B_stage1_notes.md	the OEM combo and stage-1 numbers
docs/3B_RS2_turbo_chipset.md	the RS2-turbo chipset, table by table
docs/3B_injection_path_RE.md, 3B_load_headroom_RE.md	how the 3B computes fuel and load, and where load stops
docs/3B_GT3071_plan.md	the bigger-turbo build order and the 034 kit analysis
docs/3B_what_to_log.md	the logging checklist: what to record, how, and what each log decides
docs/README.md	index of every write-up in the docs folder, grouped by ECU family
docs/ROADMAP.md	what was built and in what order